Artificial Intelligence

BS (CS) Spring 2026

Lab 07 Manual



Learning Objectives:

1. Local Search Algorithm
2. Simulated Annealing
3. Genetic Algorithm

Lab Manual

Local Search Algorithm:

Local Search Algorithms are optimization techniques used in Artificial Intelligence to find solutions by iteratively exploring neighboring options in the solution space. That basically focus on iteratively **moving** from **one solution** to a **neighboring solution**. They do this by evaluating solutions based on a objective function, which provides a measure of the solution's quality. These algorithms aim to find the best solution in the vicinity of the current state, rather than exploring the entire solution space. This local exploration strategy is what makes them particularly valuable.

Simulated Annealing

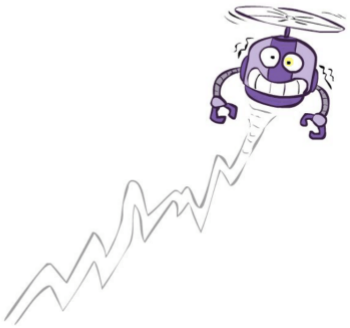
Simulated Annealing is a local search algorithm inspired by the annealing process in metallurgy, where controlled cooling of a material results in a stable crystalline structure. This algorithm introduces randomness to overcome the limitations of traditional local search methods, such as getting stuck in local optima.

How Simulated Annealing Works:

1. **Initialization:** Start with an initial solution and a high "temperature" parameter.
2. **Neighbor Selection:** Choose a random neighboring solution.
3. **Evaluation:** Compare the objective function values of the current and neighboring solutions.
    - If the new solution is better, move to it.
    - If the new solution is worse, move to it with a probability that decreases as the "temperature" lowers.
4. **Cooling Schedule:** Gradually reduce the "temperature" over iterations, limiting the acceptance of worse solutions as the algorithm progresses.
5. **Termination:** Stop when the temperature reaches a predefined minimum or no significant improvement occurs.

Pseudocode

```
function SIMULATED-ANNEALING(problem,schedule) returns a  state
current ← problem.initial-state
for t = 1 to ∞ do
    T ←schedule(t)
    if T = 0 then return current
    next ← a randomly selected successor of current
    ΔE ← next.value – current.value
    if ΔE > 0 then current ← next
              else current ← next only with probability e^(ΔE/T)
```



Key Features:

- **Escaping Local Optima**: The probabilistic acceptance of worse solutions helps avoid being trapped in local optima.
- **Controlled Exploration:** The "temperature" determines how likely worse solutions are accepted, balancing exploration and exploitation.

Advantages:

- Effective for problems with numerous local optima.
- Provides a mechanism for escaping plateaus and ridges in the search space.

Limitations:

- Performance depends on the cooling schedule and initial temperature.
- May require significant computational time for convergence.

Example Application:

In a scheduling problem, such as optimizing staff shifts, simulated annealing explores various schedules. Early iterations may accept suboptimal schedules to explore diverse possibilities, while later iterations focus on refining the solution.

Implementation

```python
import random
import math

def objective_function(route):
  # the total distance of the route
  return sum(math.sqrt((route[i] - route[i-1])**2) for i in range(len(route)))

def simulated_annealing(initial_route, temperature, cooling_rate):
  current_route = initial_route
  current_score = objective_function(current_route)
  best_route = current_route
  best_score = current_score

  while temperature > 0.1:
      new_route = current_route[:]
      i, j = random.sample(range(len(route)), 2)
      new_route[i], new_route[j] = new_route[j], new_route[i]
      new_score = objective_function(new_route)

      if new_score < current_score or random.random() < math.exp((current_score - new_score) / temperature):
          current_route = new_route
          current_score = new_score
          if new_score < best_score:
              best_route = new_route
              best_score = new_score

      temperature *= cooling_rate

  return best_route, best_score

# Example usage
route = [0, 1, 2, 3, 4]
best_route, best_score = simulated_annealing(route, 1000, 0.995)
print(f"Best route: {best_route} with score: {best_score}")
```

Genetic Algorithms

Genetic Algorithms (GAs) are local search algorithms inspired by the principles of natural selection and genetics in biology. They use a population of candidate solutions, evolve them through iterative steps, and aim to find the best or optimal solution.

"**A genetic algorithm is a search technique that mimics natural selection to find optimal solutions by iteratively refining a population of candidate solutions.**"

Understanding the basic biological principles behind GAs can help us understand how these algorithms work and help us use them more strategically.

Components of Genetic Algorithm

Genetic algorithms were inspired by evolution. These are the following components of genetic algorithm.

Population

The population in a genetic algorithm is a set of candidate solutions, often called individuals. Each individual is represented as a chromosome, which can be coded as binary strings, arrays, or other data structures.

For example, each chromosome could represent a set of input values to a function we need to optimize.

Example: Imagine trying to optimize delivery routes for trucks. Each truck has many possible routes, and you have many trucks with many stops. The sheer number of possible combinations can be overwhelming, and finding the best solution can be like searching for a needle in a haystack.

```
# Create the initial population
def create_initial_population(size, lower_bound, upper_bound):
    population = []
    for _ in range(size):
        individual = (random.uniform(lower_bound, upper_bound),
                      random.uniform(lower_bound, upper_bound),
                      random.uniform(lower_bound, upper_bound))
        population.append(individual)
    return population
```

Fitness Function

The fitness function evaluates each individual's ability to solve the problem we're interested in. It assigns a fitness value to each individual, with higher values for better solutions. The fitness function guides the algorithm toward better solutions.

For instance, in our mathematical function optimization example, the fitness function might return the value of the function for the given inputs.

For our truck routes example, the fitness function might return the speed of delivery completion.

```
# Define the fitness function (objective is to create the flattest U-shape)
def fitness_function(params):
    a, b, c = params
    if a <= 0:
        return -float('inf')  # Penalize downward facing u-shapes heavily
    vertex_x = -b / (2 * a) #x value at vertex
    vertex_y = a * (vertex_x ** 2) + b * vertex_x + c #y value at vertex
    y_left = a * (-1) ** 2 + b * (-1) + c #y-coordinate at x = -1
    y_right = a * (1) ** 2 + b * (1) + c #y-coordinate at x = 1
    curviness = abs(y_left - vertex_y) + abs(y_right - vertex_y)
    return -curviness  # Negate to minimize curviness
```

Selection Function

The selection function chooses individuals from the population to reproduce based on their fitness. Individuals with **higher fitness** are more likely to be chosen for reproduction. This mimics natural selection, where the best-adapted individuals are more likely to survive and reproduce.

There are a few different selection methods. Common selection methods include **roulette wheel selection, tournament selection,** and **rank-based selection**.

1. **Roulette wheel selection** is where individuals are selected based on a probability proportional to their fitness, similar to spinning a roulette wheel.
2. In **tournament selection** a subset of individuals is chosen at random and the fittest individual from this subset is selected.
3. **Rank-based selection** ranks individuals based on their fitness. Then selection probabilities are assigned based on these ranks rather than raw fitness values.

Each method has its own advantages and should be chosen based on the specific requirements of the problem at hand.

```
# Selection function using tournament selection
def selection(population, fitnesses, tournament_size=3):
    selected = []
    for _ in range(len(population)):
        tournament = random.sample(list(zip(population, fitnesses)),
tournament_size)
        winner = max(tournament, key=lambda x: x[1])[0]
        selected.append(winner)
    return selected
```

Crossover Function

Crossover combines information from two individuals to create offspring. The goal is to inherit beneficial traits from both parents. Common crossover techniques include **single-point crossover**, **multi-point crossover**, **uniform crossover**, and **blend crossover**.

1. **Single-point crossover** involves choosing a random point and swapping the genetic material from two parents at that point to create two offspring.
2. **Multi-point crossover** uses multiple points for swapping segments between parents, allowing for more complex genetic combinations.
3. **Uniform crossover** randomly decides which parent will contribute each gene, providing a high level of genetic diversity.
4. **Blend crossover** generates offspring by blending the genes of the parents using a random blending factor.

The decision of which technique to use should be based on the problem's complexity and the desired level of diversity in the offspring.

```
# Crossover function
def crossover(parent1, parent2):
    alpha = random.random()
    child1 = tuple(alpha * p1 + (1 - alpha) * p2 for p1, p2 in zip(parent1, parent2))
    child2 = tuple(alpha * p2 + (1 - alpha) * p1 for p1, p2 in zip(parent1, parent2))
    return child1, child2
```

Mutation Function

Mutation introduces random changes in the offspring's genetic material. This helps maintain diversity in the population and explores new areas of the solution space. Mutations can be as simple as flipping a bit in a binary string or may involve more complex changes depending on the encoding scheme.

```
# Mutation function
def mutation(individual, mutation_rate, lower_bound, upper_bound):
    individual = list(individual)
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            mutation_amount = random.uniform(-1, 1)
            individual[i] += mutation_amount
            # Ensure the individual stays within bounds
            individual[i] = max(min(individual[i], upper_bound), lower_bound)
    return tuple(individual)
```

How Genetic Algorithms Work:

1. **Initialization:** Start with a randomly generated population of candidate solutions.
2. **Evaluation:** Assess each solution using a fitness function that quantifies its quality.
3. **Selection:** Choose the best-performing solutions to act as “parents” for the next generation.
4. **Crossover (Recombination):** Combine pairs of parents to produce “offspring” by exchanging parts of their solutions.
5. **Mutation:** Introduce small, random changes in some offspring to maintain diversity in the population.
6. **Replacement:** Replace the old population with the new one and repeat the process until a termination condition is met (e.g., a maximum number of generations or a satisfactory solution).

Pseudocode

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
    new_population ← empty set
    for i = 1 to SIZE(population) do
      x ← RANDOM-SELECTION(population, FITNESS-FN)
      y ← RANDOM-SELECTION(population, FITNESS-FN)
      child ← REPRODUCE(x, y)
      if (small random probability) then child ← MUTATE(child)
      add child to new_population
    population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN

function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

Figure 4.8 A genetic algorithm. The algorithm is the same as the one diagrammed in Figure 4.6, with one variation: in this more popular version, each mating of two parents produces only one offspring, not two.

Key Features:

- Population-Based Search: Operates on multiple solutions simultaneously, encouraging diverse exploration.
- Biological Inspiration: Mimics natural selection, inheritance, and mutation to evolve solutions.

Advantages:

- Effective in exploring vast and complex solution spaces.
- Less likely to get trapped in local optima due to diverse population and mutations.

Limitations:

- Computationally expensive due to the evaluation of a large population over multiple generations.
- Requires careful tuning of parameters like population size, mutation rate, and crossover rate.

Example Application:

In optimizing delivery routes, genetic algorithms can start with a random set of possible routes, recombine high-performing routes, and mutate parts of them to find the best solution efficiently over several generations.

Comparison with Other Local Search Algorithms:

- Unlike hill-climbing, which focuses on a single path, genetic algorithms explore multiple paths in parallel.
- Unlike simulated annealing, genetic algorithms do not rely on randomness alone but use structured operations like crossover and mutation.